Syllabus: Surrogate Modeling for Process Prediction and Optimization

Build validated process predictors, learn optimization formulations and model structures, then select and revalidate operating conditions.

Course design and notes: Sunwoo Kim · Graduate entry

Course purpose

Define inputs and outputs, fit a surrogate to process-response data, and validate its predictions under independent conditions. Then assess physical consistency, represent the learned function in an optimization problem, and select operating conditions for revalidation with the reference process model.

The course covers both formulating an already-trained model for optimization and designing a model structure for that use. Prediction error, physical consistency, formulation agreement, and decision quality are evaluated separately.

Sunwoo Kim designed this eight-week, self-paced course for chemical engineers entering graduate study. Original explanations, worked readings, and runnable examples accompany verified research sources.

Audience and prerequisites

Basic Python, derivatives, matrix operations, linear algebra, and material balances are expected. LP/MILP, convexity, and KKT conditions are introduced when needed, so prior neural-network or mathematical-programming research experience is not required.

The recommended weekly workload is 90 minutes of theory, 60–90 minutes of guided coding, and 2–3 hours of reading or exercises.

Learning outcomes

By the end of the course, learners should be able to:

1. Define a process prediction task: inputs, outputs, fixed context, states, units, data splits, and valid domain.
2. Fit steady-state and dynamic surrogates, and evaluate component errors and trajectory errors using independent data and prescribed future inputs.
3. Compare unconstrained, soft-physics, and hard linear-equality models, separating prediction accuracy from physical consistency.

4. Define operating variables, an objective, and constraints; formulate a frozen ReLU model as a valid MILP and check agreement with its forward pass.
5. Select model structure and formulation for the intended use, including the conditions under which ReLU ICNN/PICNN epigraphs permit an equivalent LP problem.
6. Solve a surrogate-based operating problem and report reference-model feasibility, economic performance, solver termination, gaps, and time separately.

Eight-week sequence

Week	Topic and scope	Required output
1	**Process Prediction and Surrogate Modeling Basics.** Inputs, outputs, fixed context, units, operating domain, reference data, a regression baseline, and independent prediction checks.	A steady-state CSTR data contract, fitted regression baseline, and held-out prediction report.
2	**Learning Multicomponent Steady-State Responses with MLPs.** Three-component balances, scaling, MLP forward/backpropagation, training, component errors, and concentration consistency.	A steady-state MLP evaluated in physical units, with a hand-checked gradient update.
3	**Predicting Dynamic Responses and Rollout.** State-transition MLPs, initial states, future operating inputs, one-step prediction, and full-trajectory rollout. RNN/LSTM and encoder–decoder extensions are optional.	A comparison of one-step and trajectory errors under prescribed future inputs.
4	**Learning with Physical Constraints.** Residual losses, automatic differentiation, hard linear-equality projection, KKT derivation, rank assumptions, and remaining violations.	An MLP/soft-physics/hard-equality comparison separating accuracy from physical consistency.
5	**From a Predictor to an Optimization Formulation.** Operating variables versus fixed context, objectives and constraints, frozen ReLU graphs, binary variables, Big-M, valid bounds, and scaling.	A hand-built MILP representation checked against the frozen network forward pass.
6	**Building Formulations We Can Solve.** Bound tightening, scaling, OMLT, relaxations, solver status, gaps, computation time, and optional short-horizon extensions.	An operating optimization solve with documented bounds, termination, gaps, timing, and forward rechecks.
7	**Designing Model Structure for Optimization.** ICNN/PICNN, convexity conditions, ReLU/linear epigraphs, fixed context, inequality direction, and equivalence counterexamples.	An ICNN and valid LP on a known convex example, with explicit applicability conditions.
8	**Selecting and Revalidating Operating Conditions.** Decision-point errors, extrapolation, physical feasibility, economic performance, reference-model checks, and targeted additional data.	A reproducible prediction-to-optimization report separating errors, violations, decision quality, and computation.

Reading assignments

Week	Reading guidance
1	Week 1 reading companion; Boyd & Vandenberghe, affine mappings (optional).
2	Week 2 reading companion; selected PyTorch model, autograd, and training tutorials.
3	Week 3 reading companion: state-transition derivations, worked calculations, and evaluation code; Sutskever et al., Sections 1–2 (optional).
4	Week 4 reading companion; Raissi et al., residual-loss formulation; Chen et al., Section 3 and selected process cases.
5	Week 5 reading companion; Boyd & Vandenberghe, optimization problems (selected); Anderson et al., ReLU formulations.
6	OMLT paper and documentation; selected stronger-formulation material from Anderson et al.
7	Amos et al., Section 3 and Supplement B.
8	Revisit the model and formulation assumptions, validation evidence, and reference results from Weeks 1–7.

Core sources

- Convex optimization background: S. Boyd and L. Vandenberghe, *Convex Optimization* (2004). Read selected parts of Chapters 2–4 for convex sets, convex functions, epigraphs, and optimization problems; return to Chapter 5 for duality and KKT. **Official book and downloads** · **EE364a slides**.
- Neural-network implementation: PyTorch tutorials on **model construction**, **autograd**, and **training**.
- ICNN/PICNN: B. Amos, L. Xu, and J. Z. Kolter, *Input Convex Neural Networks*, ICML (2017). Section 3 gives architecture conditions; Supplement B gives LP inference for ReLU/linear units. **Paper** · **PDF** · **Supplement**.
- ReLU formulations: R. Anderson, J. Huchette, W. Ma, C. Tjandraatmadja, and J. P. Vielma, *Strong mixed-integer programming formulations for trained neural networks*, Mathematical Programming (2020). The public manuscript was first submitted in 2018 and revised in 2020. **Public manuscript**.
- Embedding software: F. Ceccon, J. Jalving, J. Haddad, A. Thebelt, C. Tsay, C. D. Laird, and R. Misener, *OMLT: Optimization & Machine Learning Toolkit*, JMLR 23(349), 1–8 (2022). **Paper and PDF** · **Code** · **Documentation**.
- PINN: M. Raissi, P. Perdikaris, and G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics 378, 686–707 (2019). **Published paper**. The related 2017 Part I manuscript is a freely available introduction; it is not the identical final journal version. **2017 Part I**.
- KKT-hPINN: H. Chen, G. E. Constante Flores, and C. Li, *Physics-informed neural networks with hard linear equality constraints*, Computers & Chemical Engineering 189, 108764 (2024). **Published paper** · **Public manuscript** · **Code and datasets**.

- Sequence-learning background: I. Sutskever, O. Vinyals, and Q. V. Le, *Sequence to Sequence Learning with Neural Networks*, NeurIPS (2014). Its application is translation; our process-trajectory exercises are separate educational examples. **Paper**.

Optional extensions: **DeepONet, Nature Machine Intelligence (2021)**, for input-function to output-function mappings; **KKT-Hardnet public manuscript (2025), published in C&CE (2026)**, for nonlinear equality and inequality projection.